

Name :Bhavya rana
Sap id :590034689
Batch : 15

1. Problem Statement

Write a menu-driven C program using `switch-case` to perform the following operations on a positive integer:

1. Generate the **largest possible number** using its digits.
2. Generate the **smallest possible number** using its digits.
3. **Swap the first and last digits.**
4. Check whether the number is a **palindrome**.

Each operation must be implemented using a separate user-defined function having:

- `void` return type
- No arguments
- Input taken inside the function
- Output displayed inside the function

2. Algorithm

Main Program Algorithm

1. Start.
2. Display the menu:
 - 1 → Largest possible number
 - 2 → Smallest possible number
 - 3 → Swap first and last digits
 - 4 → Check palindrome
3. Read the user's choice.
4. Use `switch-case`:
 - If choice is 1, call `largestNumber()`.
 - If choice is 2, call `smallestNumber()`.
 - If choice is 3, call `swapFirstLast()`.
 - If choice is 4, call `checkPalindrome()`.
 - Otherwise, display "Invalid choice".
5. Stop.

Algorithm for `largestNumber()`

1. Read a positive integer.

2. Create a frequency array for digits 0 to 9.
3. Extract each digit and count its frequency.
4. Arrange the digits from 9 down to 0.
5. Form the new number.
6. Display the largest possible number.

Algorithm for **smallestNumber()**

1. Read a positive integer.
2. Count the frequency of each digit.
3. Find the smallest **non-zero** digit and place it first.
4. Place all zeros immediately after the first digit.
5. Arrange the remaining digits in increasing order.
6. Display the smallest possible number.

The first digit cannot be zero because leading zeros do not form part of an integer.

Algorithm for **swapFirstLast()**

1. Read a positive integer.
2. Find the last digit using $\% 10$.
3. Find the positional divisor of the first digit.
4. Find the first digit.
5. Extract the middle portion of the number.
6. Form the new number by placing:
 - Last digit first
 - Middle digits unchanged
 - First digit last
7. Display the resulting number.

Algorithm for **checkPalindrome()**

1. Read a positive integer.
2. Store the original number.
3. Reverse the number digit by digit.
4. Compare the reversed number with the original number.
5. If both are equal, display "Palindrome number".
6. Otherwise, display "Not a palindrome number".

3. Test Case Table

Test Case	Choice	Input	Expected Output	Actual Output	Result
1	1	31052	Largest possible number = 52310	Largest possible number = 52310	Pass
2	2	31052	Smallest possible number = 10235	Smallest possible number = 10235	Pass
3	3	12345	Number after swapping = 52341	Number after swapping = 52341	Pass
4	4	12321	Palindrome number	Palindrome number	Pass
5	4	12345	Not a palindrome number	Not a palindrome number	Pass

4. C Program

```
#include <stdio.h>

/* Function declarations */
void largestNumber(void);
void smallestNumber(void);
void swapFirstLast(void);
void checkPalindrome(void);

int main(void)
{
    int choice;

    printf("1. Generate largest possible number\n");
    printf("2. Generate smallest possible number\n");
    printf("3. Swap first and last digits\n");
    printf("4. Check palindrome\n");

    printf("Enter your choice: ");
    scanf("%d", &choice);

    switch (choice)
    {
        case 1:
            largestNumber();
            break;

        case 2:
            smallestNumber();
            break;

        case 3:
            swapFirstLast();
            break;

        case 4:
            checkPalindrome();
            break;

        default:
            printf("Invalid choice\n");
    }
}
```

```

        return 0;
    }

/* Function to generate the largest possible number */
void largestNumber(void)
{
    long long n, temp, result = 0;
    int freq[10] = {0};
    int digit, i;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    if (n <= 0)
    {
        printf("Invalid input\n");
        return;
    }

    temp = n;

    while (temp > 0)
    {
        freq[temp % 10]++;
        temp = temp / 10;
    }

    for (digit = 9; digit >= 0; digit--)
    {
        for (i = 0; i < freq[digit]; i++)
        {
            result = result * 10 + digit;
        }
    }

    printf("Largest possible number = %lld\n", result);
}

/* Function to generate the smallest possible number */
void smallestNumber(void)
{
    long long n, temp, result = 0;

```

```

int freq[10] = {0};
int digit, i;

printf("Enter a positive integer: ");
scanf("%lld", &n);

if (n <= 0)
{
    printf("Invalid input\n");
    return;
}

temp = n;

while (temp > 0)
{
    freq[temp % 10]++;
    temp = temp / 10;
}

/* Find the smallest non-zero digit */
for (digit = 1; digit <= 9; digit++)
{
    if (freq[digit] > 0)
    {
        result = digit;
        freq[digit]--;
        break;
    }
}

/* Add all zeros after the first digit */
for (i = 0; i < freq[0]; i++)
{
    result = result * 10;
}

/* Add remaining digits in increasing order */
for (digit = 1; digit <= 9; digit++)
{
    for (i = 0; i < freq[digit]; i++)
    {
        result = result * 10 + digit;
    }
}

```

```
    printf("Smallest possible number = %lld\n", result);
}
```

```
/* Function to swap first and last digits */
```

```
void swapFirstLast(void)
```

```
{
    long long n, first, last;
    long long middle, divisor = 1, result;

    printf("Enter a positive integer: ");
    scanf("%lld", &n);

    if (n <= 0)
    {
        printf("Invalid input\n");
        return;
    }

    if (n < 10)
    {
        printf("Number after swapping = %lld\n", n);
        return;
    }

    while (n / divisor >= 10)
    {
        divisor = divisor * 10;
    }

    first = n / divisor;
    last = n % 10;

    middle = (n % divisor) / 10;

    result = last * divisor + middle * 10 + first;

    printf("Number after swapping = %lld\n", result);
}
```

```
/* Function to check palindrome */
```

```
void checkPalindrome(void)
```

```
{
```

```

long long n, temp, reverse = 0;

printf("Enter a positive integer: ");
scanf("%lld", &n);

if (n <= 0)
{
    printf("Invalid input\n");
    return;
}

temp = n;

while (temp > 0)
{
    reverse = reverse * 10 + temp % 10;
    temp = temp / 10;
}

if (reverse == n)
{
    printf("Palindrome number\n");
}
else
{
    printf("Not a palindrome number\n");
}
}

```

6.execution and testing

Case 1

```

[bhavya@Bhavyas-MacBook-Air ~ % clang day17.c -o day17
[bhavya@Bhavyas-MacBook-Air ~ % ./day17
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice: 1
Enter a positive integer: 31052
Largest possible number = 53210
bhavya@Bhavyas-MacBook-Air ~ % █

```

Result : pass

Case 2

```
[bhavya@Bhavyas-MacBook-Air ~ % ./day17
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice: 2
Enter a positive integer: 31052
Smallest possible number = 10235
bhavya@Bhavyas-MacBook-Air ~ %
```

Result : pass

Case 3

```
[bhavya@Bhavyas-MacBook-Air ~ % ./day17
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice: 3
Enter a positive integer: 12345
Number after swapping = 52341
bhavya@Bhavyas-MacBook-Air ~ %
```

RESULT : PASS

CASE 4

```
[bhavya@Bhavyas-MacBook-Air ~ % ./day17
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice: 4
Enter a positive integer: 12321
Palindrome number
bhavya@Bhavyas-MacBook-Air ~ %
```

RESULT : Pass

Case 5

```
[bhavya@Bhavyas-MacBook-Air ~ % ./day17
1. Generate largest possible number
2. Generate smallest possible number
3. Swap first and last digits
4. Check palindrome
Enter your choice: 4
Enter a positive integer: 12345
Not a palindrome number
bhavya@Bhavyas-MacBook-Air ~ %
```

Result : pass